

Advanced Machine learning Mastering Course

Introduced by

George Samuel

Master in computer science
Cairo University

Innovisionray.com

2024

Advanced Machine Learning 2024 by
George Samuel



Machine Learning Diploma

- 1- Statistics
- 2- Pandas

1. Pandas Basics

Import:

```
1 import pandas as pd
```

Create Series:

With default index

```
1 s = pd.Series([1, 2, 3, 4])  
2 s
```

```
0    1  
1    2  
2    3  
3    4  
dtype: int64
```

Specify the index

```
1 # Specify the indices  
2 s = pd.Series([1, 2, 3, 4], index = ["A", "B", "C", "D"])  
3 s
```

```
A    1  
B    2  
C    3  
D    4  
dtype: int64
```

Create DataFrame:

```
1 data = [[1, 444, 'abc'],
2         [2, 555, 'def'],
3         [3, 666, 'ghi'],
4         [4, 444, 'xyz']]
5 df = pd.DataFrame(data, columns=["col1", "col2", "col3"])
6 df
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

```
1 # another way
2 data = {'col1':[1,2,3,4],
3         'col2':[444,555,666,444],
4         'col3':['abc','def','ghi','xyz']}
5
6 df = pd.DataFrame(data)
7 df
```

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Rename DataFrame Columns & index:

Rename DataFrame Columns

```
1 df = pd.DataFrame([[1, 444, 'abc'],
2                     [2, 555, 'def'],
3                     [3, 666, 'ghi'],
4                     [4, 444, 'xyz']])
5 display(df)
6 columns=["col1", "col2", "col3"]
7 df.columns = columns
8 display(df)
```

	0	1	2
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Rename DataFrame index

```
1 df = pd.DataFrame([[1, 444, 'abc'],
2                     [2, 555, 'def'],
3                     [3, 666, 'ghi'],
4                     [4, 444, 'xyz']])
5 display(df)
6 index = ["row1", "row2", "row3", "row4"]
7 df.index = index
8 display(df)
```

	0	1	2
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

	0	1	2
row1	1	444	abc
row2	2	555	def
row3	3	666	ghi
row4	4	444	xyz

Pandas Dtypes:

Bool ➤ Represents Numerical datatypes with True & False values.	Int ➤ Represents Numerical datatypes with integer values.	Float ➤ Represents Numerical datatypes with continuous values.
Category ➤ Represents Categorical datatypes.	Object ➤ Is a mix of categorical datatypes & Numerical datatypes. ➤ Can carry any python object; such as, lists, tuples, strings, etc.	

Pandas Dtypes:

Get Datatypes of all columns

```
1 # Datatype of all columns  
2 df.dtypes
```

```
col1    int64  
col2    int64  
col3    object  
dtype: object
```

Get Datatype of one column

```
1 # Datatype of one column  
2 df['col1'].dtype
```

```
dtype('int64')
```

Change Datatype:

Change Datatype of one column

```
1 df["col1"] = df["col1"].astype("category")
2 df.dtypes
```

```
col1    category
col2      int64
col3      object
dtype: object
```

Change Datatypes of group of columns

```
1 cols = ["col1", "col3"]
2 df[cols] = df[cols].astype("category")
3 df.dtypes
```

```
col1    category
col2      int64
col3    category
dtype: object
```

2. EDA using Pandas

EDA using Pandas:

- **EDA** is about exploring and understanding the data and getting insights about it.
- **EDA** is short for **Exploratory Data Analysis**.
- Pandas provides built-in methods and features that helps us to answer different questions about the data.

EDA using Pandas:

Get the first n rows of DataFrame

```
1 df.head(2)
```

	col1	col2	col3
0	1	444	abc
1	2	555	def

Get the last n rows of DataFrame

```
1 df.tail(2)
```

	col1	col2	col3
2	3	666	ghi
3	4	444	xyz

Get Statistical Measures about Numerical Columns

```
1 df.describe()
```

	count	mean	std	min	25%	50%	75%	max
col2	4.0	527.25	106.274409	444.0	444.0	499.5	582.75	666.0

Get Statistical Measures about Categorical Columns

```
1 df.describe(include="category")
```

	col1
count	4
unique	4
top	1
freq	1

EDA using Pandas:

Get DataFrame Rows Names

```
1 df.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

Get DataFrame Columns Names

```
1 df.columns
```

```
Index(['col1', 'col2', 'col3'], dtype='object')
```

Get Unique Values

```
1 # get the unique values
2 df['col2'].unique()
```

```
array([444, 555, 666], dtype=int64)
```

Get Unique Values Number

```
1 # get number of unique values
2 df['col2'].nunique()
```

```
3
```

EDA using Pandas:

Get Max Value

```
1 df["col2"].max()
```

666

Get Element whose value is Max

```
1 df.col2.idxmax()
```

2

Get Min Value

```
1 df["col2"].min()
```

444

Get Element whose value is Min

```
1 df.col2.idxmin()
```

0

EDA using

Pandas:

DataFrame Basic Information

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   col1    4 non-null       category
1   col2    4 non-null       int64  
2   col3    4 non-null       object 
dtypes: category(1), int64(1), object(1)
memory usage: 400.0+ bytes
```

Unique Values Frequency

```
1 d = df['col2'].value_counts()
2 d

444    2
555    1
666    1
Name: col2, dtype: int64
```

Get Max Value of All Columns

```
1 df.max()

col2    666
col3    xyz
dtype: object
```

Summation

```
1 df['col2'].sum()

2109
```

3. Data Manipulation

Data Manipulation:

- Pandas provides built-in methods and attributes that allows you to apply different **operations over** Pandas **DataFrames** or **Series**.
- Below are the **most popular & Important attributes** that allows you to apply **data manipulation**.

Data Manipulation (Pandas to Numpy):

Convert Pandas Series into Numpy 1D-Array

```
1 df['col1'].values
```

```
[1, 2, 3, 4]  
Categories (4, int64): [1, 2, 3, 4]
```

Convert Pandas DataFrame into Numpy 2D-Array

```
1 df.values
```

```
array([[1, 444, 'abc'],  
       [2, 555, 'def'],  
       [3, 666, 'ghi'],  
       [4, 444, 'xyz']], dtype=object)
```

Data Manipulation (Replace Values):

Replace a Single Value

```
1 df.replace(555, "ali")
```

	col1	col2	col3
0	1	444	abc
1	2	ali	def
2	3	666	ghi
3	4	444	xyz

Replace Multiple Values using Dictionary

```
1 df.replace({444: "omar", "abc": 444})
```

	col1	col2	col3
0	1	omar	444
1	2	555	def
2	3	666	ghi
3	4	omar	xyz

Replace Multiple Values by One Value

```
1 df.replace([1, 666, "abc"], "aaa")
```

	col1	col2	col3
0	aaa	444	aaa
1	2	555	def
2	3	aaa	ghi
3	4	444	xyz

Data Manipulation (Mapping):

Before Mapping

1 df

	col1	col2	col3
0	1	444	abc
3	4	444	xyz
1	2	555	def
2	3	666	ghi

After Mapping

1 df.col2 = df.col2.map({444: "Fours", 555: "Fives", 666: "Sixs"})
2 df

	col1	col2	col3
0	1	Fours	abc
3	4	Fours	xyz
1	2	Fives	def
2	3	Sixs	ghi

Data Manipulation (Sorting):

Ascending sorting

```
1 sorted_df = df.sort_values(by='col1')
2 sorted_df
```

	col1	col2	col3
0	1	Fours	abc
1	2	Fives	def
2	3	Sixs	ghi
3	4	Fours	xyz

Descending sorting

```
1 sorted_df = df.sort_values(by='col1', ascending=False)
2 sorted_df
```

	col1	col2	col3
3	4	Fours	xyz
2	3	Sixs	ghi
1	2	Fives	def
0	1	Fours	abc

Data Manipulation (Apply method):

- Is a methods used to apply a certain function to each sample in the DataFrame.

Example1

```
1 def duplicate(x):
2     return x*2
3
4 df['col1'].apply(duplicate)
```

0	11
1	22
2	33
3	44

Name: col1, dtype: object

Example2

```
1 # apply built in function
2 df['col1'].apply(len)
```

0	1
1	1
2	1
3	1

Name: col1, dtype: int64

Example3

```
1 # or use lambda function
2 df['col1'].apply(lambda x: x*2)
```

0	11
1	22
2	33
3	44

Name: col1, dtype: object

Example4

```
1 df2 = pd.DataFrame([[1, 'ALI' , 3],
2                     [5, 'OMAR' , 8],
3                     [4, 'AHMED', 9]])
4 df2.apply(lambda x: x*2)
```

	0	1	2
0	2	ALIALI	6
1	10	OMAROMAR	16
2	8	AHMEDAHMED	18